

• PRODUCT LAUNCH • 2026

Nimbus — *background jobs*, finally boring

Durable task execution for product teams who would rather not babysit a queue. Submit work, get results, sleep through the night.



• WHAT WE'LL COVER

The *runsheet*

01 •

The problem Why background work quietly becomes the scariest part of a system.

02 •

The model Jobs, runs, and queues — the whole vocabulary on one page.

03 •

The proof Numbers, lifecycle, and a rollout you can plan around.



• WHY NOW

99.98%

of jobs land on the *first try* — no dead-letter archaeology, no 3 a.m. replays.



• THE NUMBERS THAT MATTER

Boring, *by the metric*

50 ms p95 cold-start before a handler runs — fast enough to sit in a request path.

99.98% of jobs succeed on the first attempt, no manual replay required.

3 regions live today for execution close to your data, with more on the way.

Three promises, *no asterisks*

Durable

Every job is persisted before it runs. A crash mid-flight resumes exactly where it stopped — never silently dropped.

Observable

One timeline per job: inputs, retries, logs, and the final result, all queryable from the dashboard or the API.

Boring

No brokers to tune, no shards to rebalance. You ship the handler; we run the infrastructure underneath it.



Three jobs *off your plate*

Persistence

Every request is written to durable storage before it runs, so a crash can never silently drop work.

Scheduling

Workers lease, run, and retry on their own — you never tune a broker or rebalance a shard.

History

Inputs, retries, logs, and results stay queryable long after the job is done.



Inline *or* in the background

Inline call

Block the request, wait for the result, return it directly. Simple, but every slow dependency becomes the user's wait.

Background job

Submit, return immediately, and let Nimbus finish the work durably. The user moves on; the result lands when it's ready.



• PICK A PLAN

Sized for *how you run*

Starter

10k jobs / month. Shared workers, 7-day history. Ideal for side projects and early prototypes.

Team

5M jobs / month. Dedicated workers, 30-day history, priority queues, and on-call alerting.

Scale

Unmetered jobs. Multi-region execution, 1-year history, custom retention, and a named reliability engineer.



A small, *senior team*

Lee Park

Founder & CEO. Built and ran job infrastructure at two prior platforms before deciding it should be boring.

Ria Okafor

Head of Reliability. Spent a decade keeping queues quiet at scale; now she keeps yours quiet too.



A small *vocabulary*

Job

One unit of work with inputs and a handler.

Run

A single attempt at a job; retries make new runs.

Queue

An ordered, durable lane jobs flow through.

Handler

Your function that turns inputs into a result.



- WHAT SHIPS IN THE BOX

The starter *project*

nimbus.toml Project
+ queue config.

handlers/ Your job
functions live here.

schemas/ Typed
inputs and results.

tests/ Replay fixtures
for every job.

- TWO HALVES OF THE SAME IDEA

Submit once, *trust it*

At submit time

The SDK validates inputs against your schema, assigns a stable job id, and persists the request before returning — so a network blip can't lose work.

At run time

A worker leases the job, runs your handler in isolation, and commits the result atomically. Crash, retry, or scale-out, the job advances exactly once.



Three roads, *one destination*

From cron

Scripts that drifted into critical infrastructure. Nimbus gives them retries, history, and alerting without a rewrite.

From a broker

Raw queues that need babysitting. Keep the mental model; drop the operational tax of brokers and consumers.

From a SaaS

A black-box runner you've outgrown. Bring handlers in-house with replayable history and no per-task surcharge.



- WHAT ACTUALLY CHANGES

Before Nimbus, *after Nimbus*

Before Hand-rolled retry loops, a dead-letter table nobody reads, and a pager that fires at 3 a.m. when a job quietly vanishes.

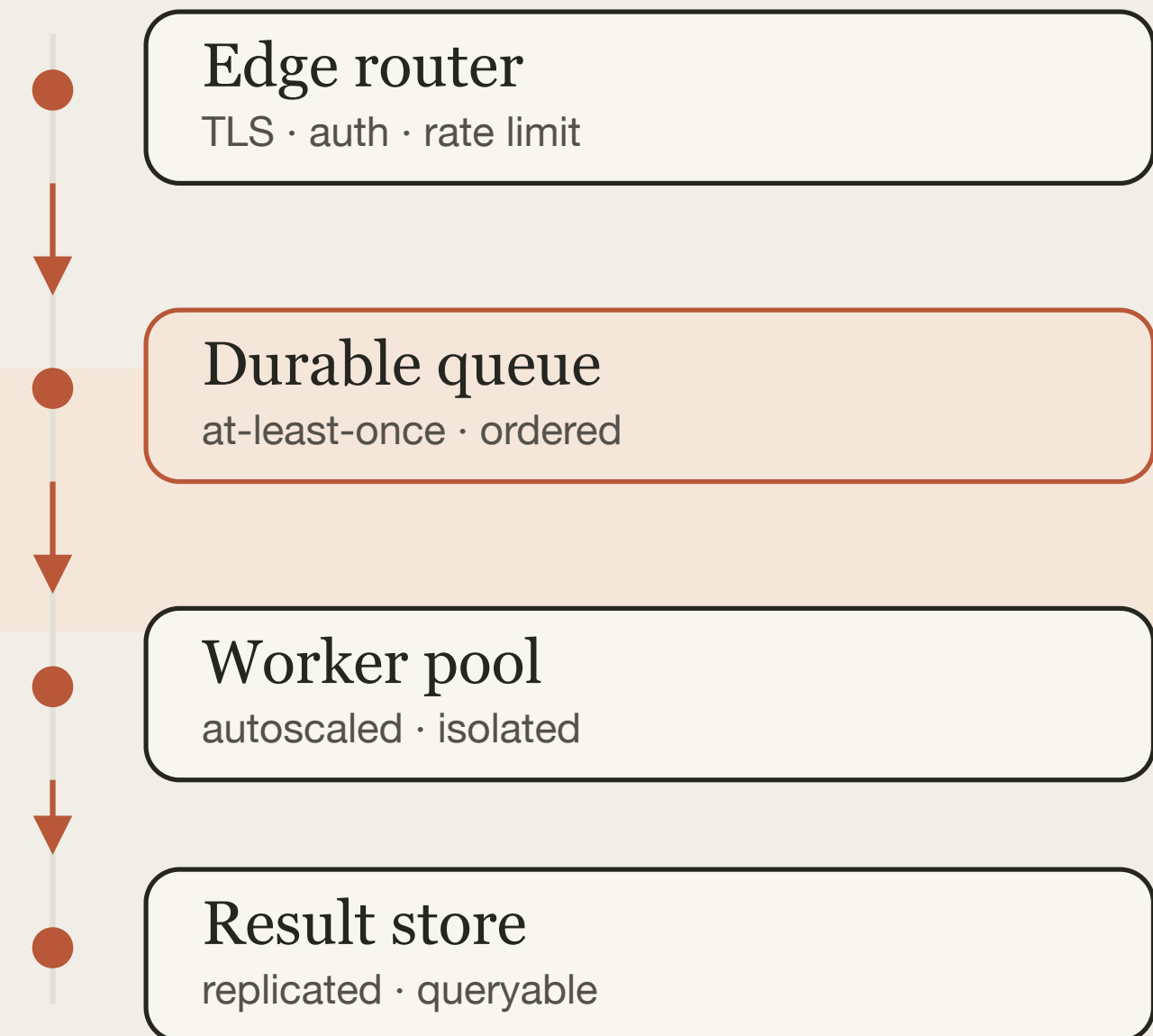
After One durable timeline per job, automatic retries with history, and a queue you genuinely stop thinking about.



A request's *short life*

Every submission walks the same four stops. The edge authenticates and rate-limits, the durable queue guarantees ordered at-least-once delivery, the worker pool runs your handler in isolation, and the result store keeps it queryable.

Persist first, run second — the rule the whole system is built around.



Live in *three moves*

Install

`npm i @nimbus/sdk` and drop your API key into the environment. No agents, no sidecars.

Define

Write a handler, give it a typed schema, and register it in `nimbus.toml`.

Submit

Call `nimbus.run()` from anywhere — the dashboard shows the job before the call even returns.



Three things you *stop worrying about*

Latency



Jobs start in under 50 ms
at p95 — fast enough to sit
in a request path.

Isolation



Each run is sandboxed;
one bad job can't take its
neighbours down.

Reach

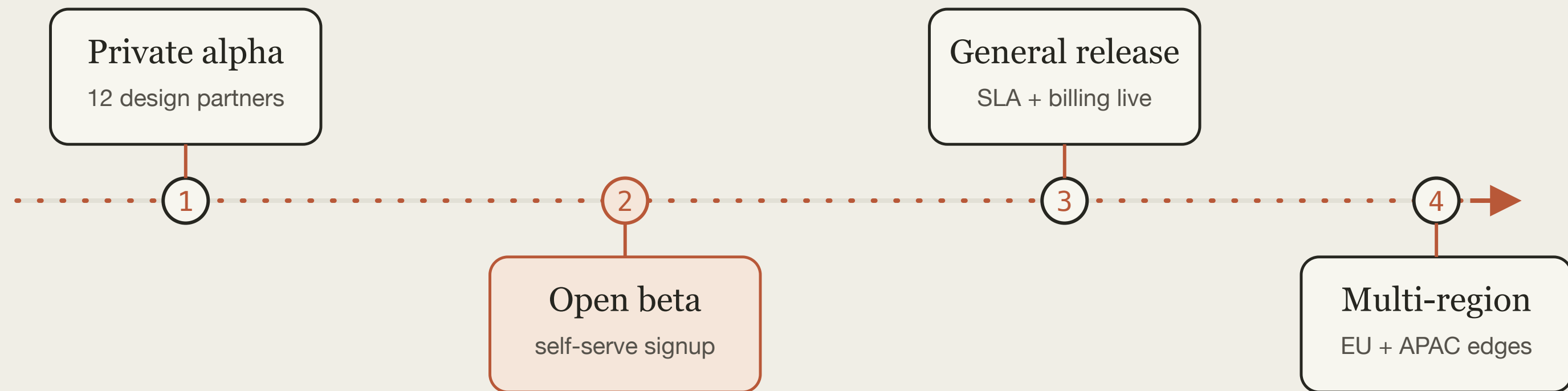


Execute close to your data
with edges in three
regions today, more on the
way.



• THE ROAD HERE AND AHEAD

A deliberate *rollout*



Open beta is live today; general release lands next quarter with billing and SLAs.

• WHEN TO REACH FOR IT

Use it *for the right work*

Great fit

Webhooks, emails, exports, billing runs, and anything that can finish slightly later without anyone waiting on the request.

Skip it

Tight synchronous reads where the caller needs the answer in the same millisecond — keep those inline, not queued.



• IN THEIR WORDS

*We deleted **600 lines** of retry glue
the week we adopted Nimbus —
and stopped getting **paged** for it.*



• THE BET

Infrastructure should *disappear*

The best background job is one nobody thinks about. Nimbus is the boring layer that lets your team spend its attention on the product, not the plumbing.



• GET STARTED

Ship something *forgettable*

Open beta is free while it lasts. Bring a handler, leave the operations to us.

